National Research University “Higher School of Economics”.
Faculty of Computer Science. Data Science.
Ordered Sets in Data Analysis.
Project of Abramov Aleksandr and Raimova Alina.

Predicting Telco Customer Churn with Lazy Formal Concept Analysis

Repository: https://github.com/TTPO100AJIEX/OSDA-LazyFCA

Table of Contents

Dataset....2
Exploratory data analysis....3
LazyFCA....5
«Vanilla» interval pattern structures....5
Computational optimizations....5
Prediction quality improvements....6
Comparison of different algorithms....7
Interpretability....9
Traditional model....9
LazyFCA....10
Individual objects....10
General principles....11
Conclusion....13

Dataset

For the task, we chose the Telco Customer Churn dataset available on Kaggle at https://www.kaggle.com/datasets/blastchar/telco-customer-churn. The dataset contains 21 features about 7043 customers of a telephone company:

- Numerical features:
  - tenure - number of months the customer has stayed with the company
  - MonthlyCharges - the amount charged to the customer monthly
  - TotalCharges - the total amount charged to the customer
- Categorical features:
  - customerID
  - gender
  - Partner - whether the customer has a partner or not
  - Dependents - whether the customer has dependents or not
  - PhoneService - whether the customer has a phone service or not
  - MultipleLines - whether the customer has multiple lines or not
  - InternetService - customer's internet service provider (DSL, Fiber optic, No)
  - OnlineSecurity - whether the customer has online security or not (Yes, No, No internet service)
  - OnlineBackup - whether the customer has online backup or not (Yes, No, No internet service)
  - DeviceProtection - whether the customer has device protection or not (Yes, No, No internet service)
  - TechSupport - whether the customer has tech support or not (Yes, No, No internet service)
  - StreamingTV - whether the customer has streaming TV or not (Yes, No, No internet service)
  - StreamingMovies - whether the customer has streaming movies or not (Yes, No, No internet service)
  - Contract - the contract term of the customer (Month-to-month, One year, Two year)
  - PaperlessBilling - whether the customer has paperless billing or not (Yes, No)
  - PaymentMethod - the customer's payment method (Electronic check, Mailed check, Bank transfer (automatic), Credit card)
  - SeniorCitizen - whether the customer is a senior citizen or not (1, 0)
- Target:
  - Churn - whether the customer churned or not (Yes or No)

The initial dataset contains 7043 objects. After the exploratory data analysis, 7032 objects remained and were split into the training set of size 6328 and a test set of size 704.

Exploratory data analysis

The source code for this section is provided in the 1__eda.ipynb notebook in the repository.

The column customerID was dropped as it consists simply of unique values with no useful information. Next, we found that the TotalCharges feature contains empty strings in 11 rows that we removed from the dataset.

Subsequently, we discovered an interesting correlation between the features PhoneService, InternetService, and the features MultipleLines, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, and StreamingMovies, shown in Figure 1. When the PhoneService has a value 'No', MultipleLines always has a value 'No Phone Service' and vice versa. Similarly, the feature 'InternetService' induces the value 'No Internet Service' in six other features. Therefore, we decided to replace the values 'No Phone Service' and 'No Internet Service' in all features with a simple 'No'.

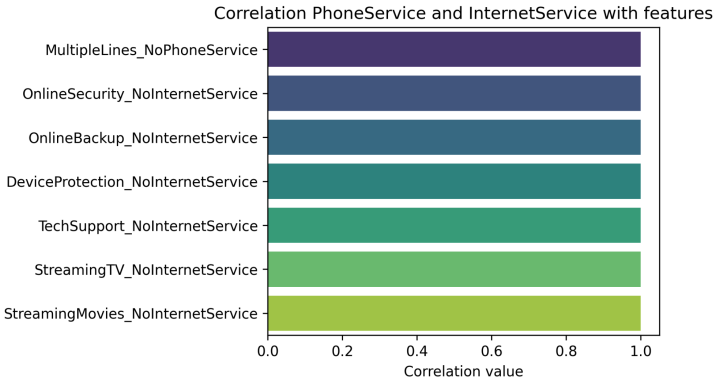

Figure 1. Correlation between features' values

Going further, we found several duplicates in the dataset with both matching and varying target values, which we explore in the pictures/duplicates.pdf file in the repository. Nonetheless, we left these rows in the dataset.

Considering the target variable, it is hugely unbalanced, as demonstrated in Figure 2. Based on this, we empirically estimate the coefficient to define negative classes at approximately 2.76.

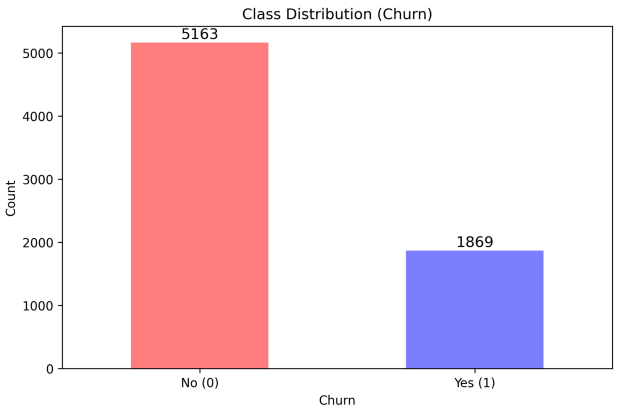

Figure 2. Class distribution

Additionally, considering the distributions of numerical features (Figure 3), we discovered that tenure roughly defines the positive class with values ranging from 0 to 20, and MonthlyCharges, ranging from 0 to 40, provides a rough definition for the negative class, yet no noticeable disparity in the distribution after 40 is observed. TotalCharges, on the other hand, has an identical distribution for both classes, giving no obvious insights for classification.

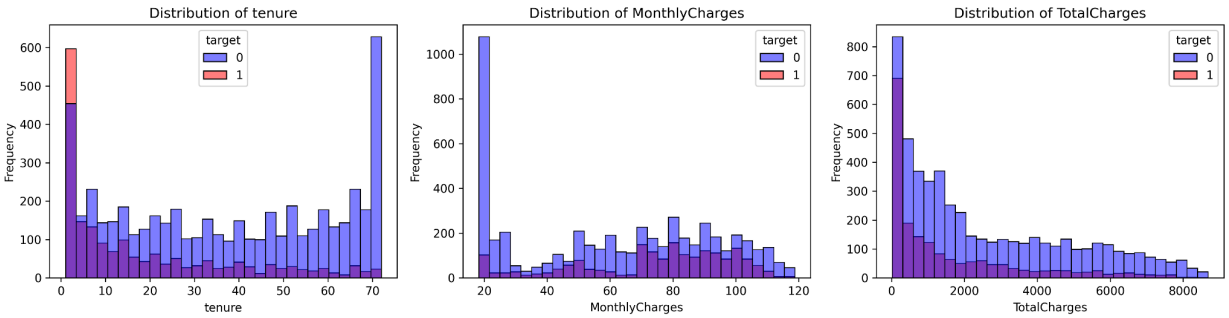

Figure 3. Distributions for numerical features

Finally, the analysis revealed correlations between the target variable and some features. As illustrated in Figure 4, tenure, Contract, and TotalCharges demonstrated the most significant impact on the target. These results may give vital insights into the data for the interpretability of the models.

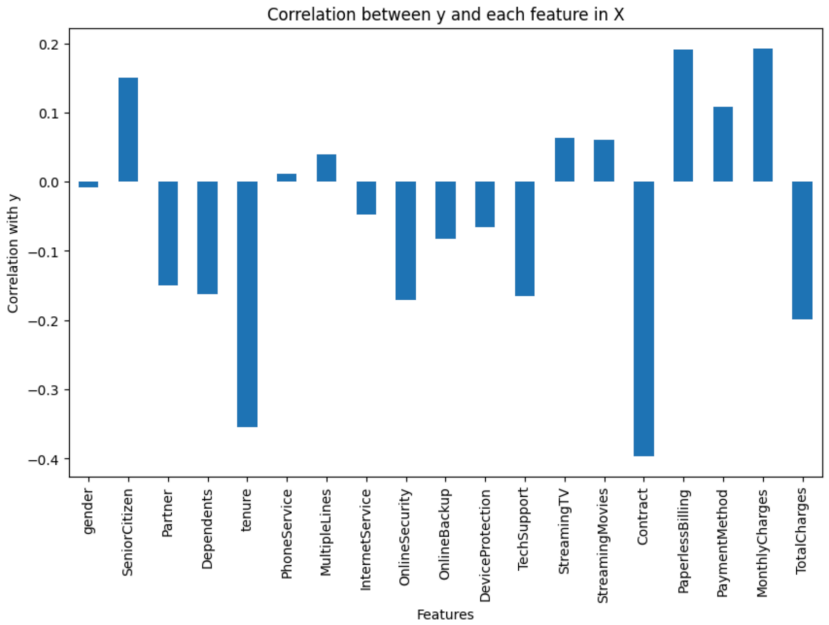

Figure 4. Correlations between the target and dataset features

LazyFCA

The source code for this section is provided in the 3__ips.ipynb notebook in the repository.

«Vanilla» interval pattern structures

For LazyFCA, we used the interval pattern structures defined as follows: $[a_1, b_1] \sqcap [a_2, b_2] = [min(a_1, a_2),\ max(b_1, b_2)]$. The features tenure, MonthlyCharges, and TotalCharges were processed as numerical, while the others were considered categorical and binarized accordingly without dropping any values, which produced 36 binary features.

With this, the "vanilla" implementation from the provided notebook showed a running time estimate of 24 hours, which we considered unacceptable. So, with the first set of changes to the algorithm, we aimed to increase the speed of calculations.

Computational optimizations

To optimize the "vanilla" implementation, we pursued three principal ideas:

1. Replace slow pandas with fast vectorized numpy wherever possible, especially in the main part of the algorithm, where it computes the number of supporters and opposers for a hypothesis. To achieve this, we turned the representation of numerical intersection as a list of tuples into two numpy arrays with minimums and maximums of respective features.
2. Do as many calculations in 'fit' as possible. Specifically, we moved the code splitting the dataset into positive and negative objects, binary and negative parts from classify_sample to fit, which also contributed to escaping slow pandas operations.
3. Use joblib to compute the predictions for different objects in parallel and utilize all available computing resources.

With these optimizations, we obtained a massive 200-fold improvement in speed, reducing the runtime from 24 hours to 6 minutes. Consequently, we obtained the first result, which we consider the predictions of the "vanilla" algorithm, as no meaningful changes that might have affected the results have been made at this stage. The estimated F1-score of these predictions was 0.33, with the model noticeably preferring the majority class in its predictions.

Before diving into qualitative improvements of the approach, we had an additional interesting idea to cache intermediate results of computing positive and negative classifiers. We assumed that the number of unique intersections of two objects' features over the entire dataset must be relatively small, yet calculating the number of supporters and opposers for those hypotheses is a very computationally expensive operation. With these ideas, we decided to use a Redis database to cache the results of those

computations. Unfortunately, this did not give the improvement we expected on the first run, increasing the running time up to 12 minutes. Anyway, the second launch that operated completely from the cache finished the work in 2.5 minutes, which is a noticeable advancement. Although the result is not useful for this research, it might be an interesting optimization idea to use in production, where the requests to the model are expected to be very repetitive.

Prediction quality improvements

Talking about qualitative improvements, we introduced five hyperparameters to define positive and negative classifiers and weight them for the final prediction. For the definition of a positive classifier, we required at least a given number of supporters (min_pos_for_pos_clas) with the ratio of opposers to supporters (pos_coef_for_pos_clas) not exceeding a fixed threshold. Similarly, two hyperparameters (min_neg_for_neg_clas, neg_coef_for_neg_clas) were introduced into the definition of a negative classifier. The final hyperparameter (pos_clas_coef) was devoted to weighing the number of positive classifiers in the final prediction, showing the maximal ratio of the number of negative classifiers to the number of positive classifiers when the object is classified as positive. Additionally, for better consistency with traditional classification models, we transformed the output from a single class index to a list of class probabilities computed as the ratio of the classifiers supporting each class.

The obvious selection of the values for those hyperparameters is based on the class imbalance ratio in the original dataset: $min_pos_for_pos_clas = 2.75$, $min_neg_for_neg_clas = \frac{1}{2.75}$, which produced better predictions than before with an F1-score of 0.4.

To tune the results, we ran two iterations of an exhaustive grid search for the best values of these five hyperparameters, as shown in Table 1. The options for the second iteration were selected around the best result of the first search to reduce the steps and better approximate the true optimal values. The full reports of classification quality with different values of hyperparameters are available in selection_results.csv and selection_results_2.csv in the repository.

Table 1. Values of LazyFCA hyperparameters checked with grid search

Hyperparemeter	First grid search values	Second grid search values	Resulting best value
min_pos_for_pos_clas	0, 5, 10, 25, 50, 100, 250, 500	3, 4, 5, 6, 7	5
pos_coef_for_pos_clas	0.1, 0.25, 0.5, 0.75, 1.0, 2.5, 5.0, 7.5, 10.0	2.0, 2.25, 2.5, 2.75, 3.0	2.75
min_neg_for_neg_clas	0, 5, 10, 25, 50, 100, 250, 500, 1000, 1500	7, 10, 13, 15, 17	10
neg_coef_for_neg_clas	0.1, 0.25, 0.5, 0.75, 1.0, 2.5, 5.0, 7.5, 10.0	0.15, 0.2, 0.25, 0.3, 0.35, 0.4	0.25
pos_clas_coef	0.5, 1.0, 2.0, 2.5, 3.0, 3.5, 4.0, 5.0	0.5, 0.75, 1.0, 1.5, 2.0	1.0

The final version of the algorithm with the best values of the hyperparameters showed an F1-score of 0.63, which we believe is around the state-of-the-art of what one can get for this dataset.

Comparison of different algorithms

The source code for this section is provided in the 2__traditional.ipynb notebook in the repository.

For comparison, we trained some standard models with their best hyperparameters selected with grid search, and preprocessed the features in two different ways for various types of models. For KNN, Logistic Regression, SVM, Gradient Boosting, and MLP, the categorical features were encoded with one-hot, dropping the first category, and the numerical features were normalized with the StandardScaler. For Naive Bayes, Decision Tree, Random Forest, XGBoost, and CatBoost, the categorical features were similarly encoded with one-hot, but the numerical features were not processed in any way.

To obtain the best hyperparameters for each model, we used GridSearch with the values presented in Table 2. The hyperparameter ranges were selected based on commonly used values from the literature to ensure a comprehensive exploration of the parameter space for each model.

Table 2. Values of hyperparameters for standard models processed with the grid search

Model	Hyperparemeter	Grid search values	Best value
KNN	n_neighbors	10, 30, 50, 70	70
	weights	'uniform', 'distance'	uniform
Naive Bayes	alpha	0.1, 1.0, 10.0	10.0
Logistic Regression	C	0.1, 1.0, 10.0	1.0
	penalty	'l1', 'l2', 'none'	l2
	class_weight	None, 'balanced'	None
SVM	C	0.1, 1.0, 10.0	10.0
	kernel	'linear', 'rbf'	linear
Decision Tree	max_depth	None, 4, 8, 16	4
	min_samples_split	2, 10, 50	2
	criterion	'gini', 'entropy', 'log_loss'	gini
Random Forest	n_estimators	100, 300, 1000	100
	max_depth	None, 5, 10, 15	10
	min_samples_split	2, 10, 50	50
	criterion	'gini', 'entropy', 'log_loss'	log_loss
Gradient Boosting	n_estimators	100, 500, 1000	100
	learning_rate	0.001, 0.05, 0.1	0.05
	max_depth	3, 5, 10	3
	min_samples_split	2, 10, 50	2
XGBoost	n_estimators	20, 50, 100	100
	max_depth	3, 4, 5, 8	4
	learning_rate	0.05, 0.1	0.05
CatBoost	iterations	100, 200, 400	100
	depth	3, 4, 5, 8	4
	learning_rate	0.01, 0.05, 0.1	0.1
MLP	hidden_layer_sizes	(100,), (200,)	(100,)
	activation	'identity', 'logistic', 'tanh', 'relu'	logistic
	learning_rate_init	0.001, 0.01, 0.05	0.001

For each model, we computed the primary classification quality metrics (F1-score, accuracy, and AUC-ROC) presented in Table 3.

Table 3. Classification quality metrics of different models

Model	F1 score	Accuracy	AUC – ROC
«Vanilla» IPS	0.33	0.77	0.59
IPS	0.40	0.77	0.83
Best IPS	0.63	0.76	0.82
KNN	0.59	0.8	0.81
Naive Bayes	0.59	0.76	0.81
Logistic Regression	0.59	0.8	0.82
SVM	0.58	0.8	0.81
Decision Tree	0.50	0.79	0.81
Random Forest	0.57	0.79	0.82
Gradient Boosting	0.56	0.79	0.83
XGBoost	0.58	0.8	0.82
CatBoost	0.58	0.8	0.82
MLP	0.55	0.75	0.76

Clearly, the best standard model is the logistic regression, yet we consider LazyFCA with the best hyperparameters to produce the best result overall, as, despite the lower accuracy, it shows a higher F1-score, indicating the prevalent robustness of the results.

Interpretability

The source code for this section is provided in the 4__interpretability.ipynb notebook in the repository.

Traditional model

To assess the interpretability of the Decision Tree, we visualize its structure in Figure 5: the root node splits on "$tenure \leq 16.5$", and subsequent nodes make decisions based on features like InternetService, Contract type, and TotalCharges. Each node displays the splitting criterion, Gini impurity, the number of samples, and class counts, illustrating how customer attributes influence their categorization. The tree reveals that low tenure and having fiber optic internet increase the likelihood of classifying customers as churned, while customers with long tenure and two-year contracts are likely to stay with the company.

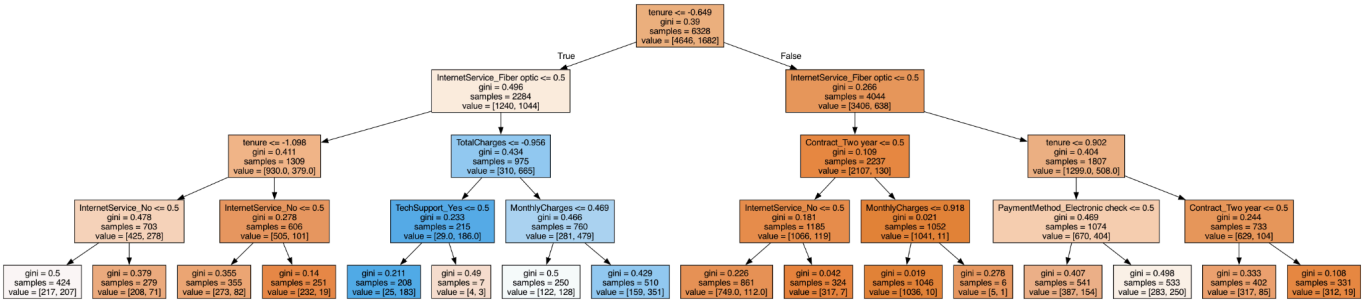

Figure 5. Visualization of a decision tree

LazyFCA

To ensure interpretability of the FCA-based classification, the work was carried out in two directions: understanding the predictions for individual objects and extracting general principles of the model's decision-making.

Individual objects

The primary framework to explain the predictions for individual objects was extracting all positive and negative classifiers from the model and reducing them to the set of the most general hypotheses. Important to note that the reduction was carried out based solely on the binary features, with numerical parts being respectively merged by subsumption. As a result, we reduced an average of 3,000 classifiers of a single object to around 30, which we believe contain the most information about the prediction and can be explored by hand. To get extra information, we also calculate the "weights" – numbers of supporters for each resulting minimal hypothesis, which lets us focus on the most important patterns found by the model.

To reinforce the results and highlight the most important features for the prediction, we also explore the features that influence the classifiers the most. For the binary features, we straightforwardly compute how many times they appear in the hypotheses, and for the numerical ones, we construct the histograms, showing the number of hypotheses that include each value of the feature.

First, let's look at an object that was correctly and confidently classified as positive. The raw artifacts, as described previously, are presented in the repository in the "positive_object" folder. From the minimal hypotheses of positive (positive_classifiers/minimal_hypotheses.csv) and negative (negative_classifiers/minimal_hypotheses.csv) classifiers, one can clearly extract several distinctions that explain the reason for the model's prediction:

- There are multiple times more positive classifiers than negative ones, and their "weights" are a lot more significant.

- The boundaries of intervals for “tenure”, especially the left one, are clearly shifted towards smaller values for positive classifiers, indicating that the customer is more likely to leave the company due to joining relatively recently.
- The feature “Contract_Month-to-month” appears primarily in the most compelling positive classifiers and in only one negative classifier, indicating that the customer in question having a month-to-month contract is a strong indicator that he is likely to churn.
- Considering the classifiers with smaller weights, “PaperlessBilling: Yes”, “OnlineSecurity: No”, “StreamingMovies: Yes”, and “InternetService: Fiber optic” also tend to appear mostly in positive hypotheses and, thus, provide additional evidence for the customer to churn.

The occurrences of the features in the hypotheses (binary_features.csv and numeric_features.png) also support these ideas and provide extra hints into the model’s decision. For example, big monthly charges and small total charges mostly prevail in the positive classifiers.

Concisely, the object under consideration was classified as positive due to having a month-to-month contract, small tenure, small total charges, and large monthly charges. Opting into paperless billing and fiber optic internet, while declining online security, were additional factors supporting that the customer is likely to churn.

Then, we similarly explore the artifacts of classifying a negative object that are published in the “negative_object” directory. One can clearly see the virtual absence of positive classifiers and a number of common patterns within the negative ones. Most importantly, the customer is not a senior citizen, has no device protection, online backup, or streaming TV, and has relatively high total charges, which were the main factors contributing to its classification as negative, meaning the unlikelihood of leaving the company.

The most interesting insights, though, were for an object where the model was not so decisive with the prediction. These are presented in the “neutral_object” directory. Here, most features contribute to either positive or negative classifiers, and rarely to both simultaneously. Based on the results, the customer under consideration is likely to stay with the company because she has some additional services enabled (tech support, online security), uses the bank transfer payment method, and has a relatively high tenure. On the other hand, being a senior citizen, having a month-to-month contract with big monthly charges, paperless billing, and fiber optic internet service are the indicators that the customer might tend to churn.

General principles

The other direction of work aimed at extracting the general principles of the model’s decision-making without focusing on individual objects. For this, we implemented CbO on the binary

features to extract all hypotheses that might be interesting for the model's explainability. The full set of formal concepts, however, is too large to be computed in full and, especially, to be analyzed by hand. So, to reduce the final output, we ran CbO on features and stopped developing a subtree preemptively when one of the two conditions was met:

1. The formal extent contained less than 25% of objects in the context. We consider these concepts too specific for interpretability, discard them from the output, and stop developing the subtree in CbO, as adding more features to the intent has no way of increasing the extent back above the threshold.
2. The formal concept satisfies the conditions for a classifier as defined by the best hyperparameters of the model obtained previously. We assume that developing the subtree by adding more features to the intent will not break the property, and the root is strictly a more general classifier than any of the ones obtained further in the subtree.

With these conditions, we aimed to find only the most general classifiers that, simultaneously, are not too specific to be uninterpretable. As a result, we obtained 64 negative concepts listed in the repository in the cbo/negative_hypotheses.txt file, and 36 positive hypotheses outlined in cbo/positive_hypotheses.txt, respectively. The most interesting ones are listed in Tables 4 and 5.

Table 4. Positive intents (define the customers that are likely to churn)

Intent	Explanation
Contract: Month-to-month	As we have already seen previously, customers with a short, month-to-month contract are more likely to leave.
PaperlessBilling: Yes	Probably, paperless billing is not very convenient in the modern day, so these customers are more likely to switch to a more suitable option.
PaymentMethod: Electronic check	Interestingly, an electronic check is the only payment method found in positive intents. Probably, this is also the least convenient option.
SeniorCitizen: 1	Senior citizens are likely to churn. We have no good explanation for this phenomenon besides those customers passing away 🙁
gender: Female	Females and people without close people are likely to churn. This is probably related to some behavioural aspects: these people are not settled in life, are still looking for better options, and are, thus, more likely to switch.
Partner: No	
Dependents: No	
OnlineSecurity: No	The customers without extra features are likely to churn, which is a relatively expected pattern.
TechSupport: No	
DeviceProtection: No	
OnlineBackup: No	
StreamingTV: Yes	Interestingly, the customers with these two extra features enabled are likely to leave. We have two plausible explanations for this: 1. Easy internet access makes streaming TV and movies irrelevant in the modern world, so people are likely to unsubscribe from these services. 2. These services are of poor quality, which makes the customers disappointed and look for other options.
StreamingMovies: Yes	

Table 5. Negative intents (define the customers that are likely to stay)

Intent	Explanation
Contract: One year	Expectedly, customers with longer contracts are more likely to stay.
Contract: Two-year	
PaperlessBilling: No	Opposite to one of the positive intents, convenient billing without any paperwork increases the likelihood of the customer staying.
PaymentMethod: Bank transfer	Similarly, all payment methods except electronic checks are factors towards the customer staying with the company.
PaymentMethod: Credit card	
PaymentMethod: Mailed check	
Partner: Yes	Again, opposite to the positive intents, having close people to care about improves the likelihood of the customer staying with the company.
Dependents: Yes	
InternetService: DSL	Expectedly, having extra features increases the satisfaction of the customers. Interestingly, though, online backup is relevant only to non-senior citizens, which may be due to the feature being mostly useful in the modern technological world.
TechSupport: Yes	
OnlineSecurity: Yes	
OnlineBackup: Yes, SeniorCitizen: 0	
InternetService: No, OnlineSecurity: No, PhoneService: Yes, OnlineBackup: No, StreamingMovies: No, StreamingTV: No, TechSupport: No, DeviceProtection: No	Interestingly, having no extra features besides the phone service itself makes people stay. Probably, these are the minimal-package customers looking to get the specific value they need for the lowest possible price. Non-senior citizens, though, tend to stay with only some of the extra features disabled: streaming movies, streaming TV, and device protection. This is also easily explainable: younger people do not need these features and do not want to pay for them.
SeniorCitizen: 0, StreamingMovies: No, StreamingTV: No, DeviceProtection: No	

Overall, the explanations are consistent with our understanding of the operation of a telephone company and highlight realistic patterns in the data. Moreover, the results correlate with the exploratory analysis of the dataset, which we consider an important outcome that increases our trust in the model's predictions.

Conclusion

In conclusion, this project successfully applied Lazy Formal Concept Analysis (LazyFCA) to predict customer churn in the Telco dataset, achieving an F1-score of 0.63 - comparable to or exceeding

traditional machine learning models. Through computational optimizations, the algorithm's runtime was drastically reduced from 24 hours to 6 minutes, making it practical for use. The model also demonstrated strong interpretability, allowing for clear insights into individual predictions and general decision-making patterns, such as the significance of contract type and payment methods. Overall, LazyFCA proved to be a powerful and explainable alternative for classification tasks, particularly in imbalanced datasets.